

Central Superstore

SQL Data Warehouse & Business Analytics

Portfolio Project Documentation

Prepared by: Abdelrahman Ahmed

University: Minia National University

Department: Artificial Intelligence

1. Project Overview

This project focuses on transforming raw Superstore sales data into a structured SQL Server data warehouse and using SQL to generate meaningful business insights.

The project follows a simplified ETL process:

Raw Data → Data Warehouse → Business Analytics

The source data was originally provided in an Excel file containing sales transactions from the Central region. The data was imported into SQL Server, transformed into a Star Schema, and analyzed using SQL queries.

The final database contains customer, product, location, date, shipping, and sales information that can be used for business reporting and analysis.

2. Business Problem

The raw sales data contains detailed transaction-level information, but analyzing the data directly can make it difficult to identify important business patterns.

The main objective of this project is to analyze the sales data and answer important business questions such as:

What are the total sales, profit, and profit margin?

Which categories and sub-categories generate the most sales and profit?

Which customers generate the highest sales?

Which products generate high sales but low or negative profit?

How do sales and profit change over time?

Which customer groups generate the most revenue?

How does discount level relate to profitability?

Which states and cities perform well financially?

How frequently do customers place orders?

What is the shipping performance of different shipping modes?

How does yearly sales performance change over time?

The project also demonstrates how SQL can be used not only for querying data, but also for building a structured analytical database and creating reusable database objects.

3. Dataset

The source dataset used in this project is a Superstore sales dataset focused on the Central region. The original data was provided in an Excel workbook and contains transaction-level sales information.

The dataset contains **2,323 transaction rows** and **21 columns**. Each row represents a sales transaction line and includes information about the order, customer, product, location, shipping method, sales, quantity, discount, and profit.

Dataset Summary

Item	Value
Source File	Central_Superstore.xlsx
Source Sheet	Central_Region
Region	Central
Total Rows	2,323
Total Columns	21
Unique Row IDs	2,323
Unique Orders	1,175
Unique Customers	629
Distinct Product IDs	1,310
Distinct Order Dates	720

The original dataset contains the following 21 columns:

Row ID, Order ID, Order Date, Ship Date, Ship Mode, Customer ID, Customer Name, Segment, Country, City, State, Postal Code, Region, Product ID, Category, Sub-Category, Product Name, Sales, Quantity, Discount, Profit.

The dataset provides the main information required to build the analytical database and answer business questions related to sales performance, customer behavior, product profitability, geographical performance, shipping, and time-based trends.

During the data preparation process, an important issue was identified in the Product ID field. Some Product IDs appeared more than once with different Product Names in the source data. As a result, the final Products dimension contains **1,326 records**, compared with **1,310 distinct Product IDs** in the original dataset.

To avoid incorrect product matching during the warehouse loading process, the product lookup was performed using both **Product ID** and **Product Name**. This ensured that all **2,323 source transactions** were correctly matched and loaded into the FactSales table.

4. Data Preparation & Import

The original Superstore dataset was provided as an Excel workbook. The first step was to prepare the source data for loading into SQL Server.

The Excel file contained the Central_Region sheet with 2,323 transaction records. Since the Excel provider required for direct SQL Server import was not available in the environment, the dataset was converted from Excel to a **UTF-8 CSV file**.

The CSV file was then imported into SQL Server using SQL Server Management Studio (SSMS) through the **Import Flat File** workflow.

The imported data was stored in a staging table named:

RawSuperstore

This table represents the **raw data layer** of the project. The original values were kept in this table so that the final data warehouse could be compared against the source data during validation.

Raw Data Structure

The RawSuperstore table contains the same 21 business columns from the original dataset, with spaces converted to underscores to make them easier to work with in SQL Server.

The main columns include:

Row_ID — unique identifier for each transaction line.

Order_ID — identifier of the customer order.

Order_Date — date when the order was placed.

Ship_Date — date when the order was shipped.

Ship_Mode — shipping method used for the order.

Customer_ID — source-system customer identifier.

Customer_Name — customer name.

Segment — customer segment.

Country, City, State, Postal_Code, Region — geographical information.

Product_ID — source-system product identifier.

Category and Sub_Category — product classification.

Product_Name — product description.

Sales — sales amount.

Quantity — number of units sold.

Discount — discount applied to the transaction.

Profit — profit generated by the transaction.

Initial Data Validation

After importing the data into RawSuperstore, several checks were performed to verify that the import was successful.

The table contained:

2,323 total rows

2,323 unique Row IDs

1,175 distinct orders

629 distinct customers

These checks confirmed that the source data was successfully imported without losing transaction records.

The Postal_Code column was stored as a text-based field because postal codes are identifiers rather than numerical measures and may contain leading zeros.

The RawSuperstore table was intentionally kept separate from the final warehouse tables. This allowed the project to follow a clear data flow:

Excel → CSV → RawSuperstore → Dimension Tables → FactSales → Business Analytics

This structure also made it possible to perform reconciliation checks between the original imported data and the final **FactSales** table.

5. Data Warehouse Architecture & Star Schema

The raw data in RawSuperstore contains all information in a single table. Although this structure is useful for storing the original data, it is not ideal for analytical queries because customer, product, location, date, and shipping information are repeated across many transaction records.

To improve the structure and make the data easier to analyze, the raw data was transformed into a **Star Schema**.

The Star Schema separates descriptive information into dimension tables and keeps transaction measures in a central fact table.

The final data warehouse consists of the following tables:

Customer

Products

Locations

Dates

ShipMode

FactSales

The FactSales table is the central fact table and is connected to the five dimension tables through foreign keys.

Dimension Tables

Customer

The Customer table stores customer-related information, including:

CustomerKey

CustomerID

CustomerName

Segment

The CustomerKey is used as the warehouse key and connects customers to their transactions in FactSales.

Products

The Products table stores product information, including:

ProductKey

ProductID

Category

SubCategory

ProductName

This table allows the analysis of sales and profit by product, category, and sub-category.

Locations

The Locations table stores geographical information, including:

LocationKey

Country

City

State

PostalCode

Region

This table is used to analyze sales and profitability across different geographical areas.

Dates

The Dates table is a date dimension that provides calendar attributes for each order date.

It contains:

DateKey

FullDate

Year

Quarter

Month

MonthName

Day

DayName

The date dimension makes time-based analysis easier, such as monthly and yearly sales trends.

ShipMode

The ShipMode table stores the available shipping methods, including:

ShipModeKey

ShipMode

It is used to analyze shipping performance and average delivery time by shipping method.

Fact Table

FactSales

The FactSales table is the central table of the warehouse. It stores transaction-level business measures and foreign keys pointing to the dimension tables.

The main columns include:

SalesKey

RowID

OrderID

CustomerKey

ProductKey

LocationKey

DateKey

ShipModeKey

ShipDate

Sales

Quantity

Discount

Profit

The fact table stores the numerical measures required for business analysis, including Sales, Quantity, Discount, and Profit.

Star Schema Relationships

The relationships between the fact and dimension tables are:

Customer.CustomerKey → FactSales.CustomerKey

Products.ProductKey → FactSales.ProductKey

Locations.LocationKey → FactSales.LocationKey

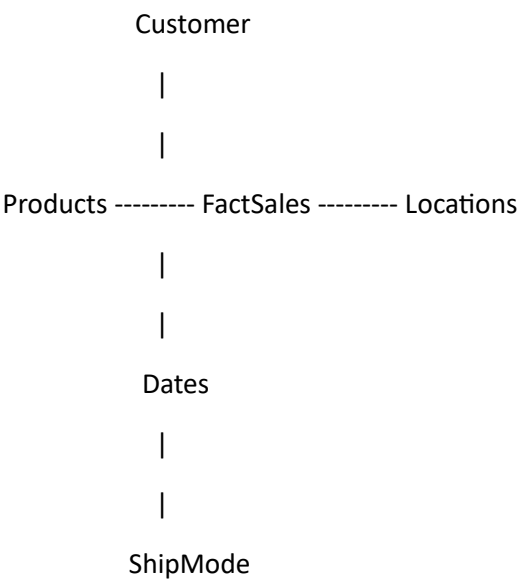
Dates.DateKey → FactSales.DateKey

ShipMode.ShipModeKey → FactSales.ShipModeKey

This structure creates a star-shaped data model where FactSales is at the center and the dimension tables surround it.

The Star Schema improves analytical queries by separating business descriptions from transaction measures and reducing unnecessary repetition of descriptive data.

Final Warehouse Structure



The resulting architecture provides a clear separation between the raw data layer and the analytical warehouse layer:

RawSuperstore → Dimensions + FactSales → Business Analytics

6. Keys and Fact Table Grain

The data warehouse uses different types of keys to identify records and establish relationships between the fact and dimension tables.

Source IDs and Surrogate Keys

The original dataset contains business identifiers such as CustomerID and ProductID. These IDs come directly from the source system.

In the data warehouse, separate surrogate keys were created for the dimension tables, such as:

CustomerKey

ProductKey

LocationKey

DateKey

ShipModeKey

The dimension keys are used to connect the dimension tables with the FactSales table.

For example:

Customer.CustomerKey → FactSales.CustomerKey

The surrogate keys were generated automatically using SQL Server IDENTITY for the dimension tables.

Fact Table Grain

The grain of the FactSales table is defined as:

One row represents one transaction line from the source dataset.

The source dataset contains **2,323 transaction rows**, so the final FactSales table also contains **2,323 rows**.

However, the dataset contains only **1,175 unique Order IDs**. This is because a single order can contain multiple products or transaction lines.

For example, one order could contain:

Order 1001

— **Product A**

— **Product B**

— **Product C**

This represents one order but three transaction lines.

Therefore, when calculating the number of orders, the project uses:

COUNT(DISTINCT OrderID)

instead of simply counting rows in FactSales.

Product Identification Issue

During the data preparation process, an important issue was discovered in the source data.

Some ProductID values appeared more than once with different product names. Therefore, ProductID alone could not always uniquely identify a product in the source dataset.

To correctly match products during the loading of FactSales, the lookup used both:

Product_ID + Product_Name

This ensured that each source transaction was matched to the correct product record.

Why This Design Was Used

This design allows the warehouse to:

Preserve the original transaction-level detail.

Avoid storing repeated descriptive information in the fact table.

Connect transactions to customers, products, locations, dates, and shipping methods.

Perform accurate order-level and transaction-level analysis.

Support flexible business analytics using SQL.

The final fact table therefore provides a reliable foundation for analyzing sales, profit, quantity, discounts, customers, products, locations, dates, and shipping performance.

7. ETL Process

The project follows a simplified ETL process to transform the raw Superstore data into a structured analytical data warehouse.

ETL stands for:

Extract → Transform → Load

The process used in this project can be summarized as:

Excel → CSV → RawSuperstore → Dimension Tables → FactSales → Business Analytics

Extract

The original data was provided in an Excel workbook named Central_Superstore.xlsx.

The data was extracted from the Central_Region sheet, which contains 2,323 transaction records and 21 columns.

Because the Excel provider required for direct SQL Server import was not available, the source file was converted into a UTF-8 CSV file before importing it into SQL Server.

The CSV file was then imported into a staging table named RawSuperstore.

Transform

After importing the raw data, the information was transformed into separate dimension tables according to the Star Schema design.

The following dimension tables were created:

Customer

Products

Locations

Dates

ShipMode

Each dimension contains descriptive information related to a specific business area.

For example, customer information was separated from transaction data and stored in the Customer dimension, while product information was stored in the Products dimension.

A separate Dates dimension was also created to provide additional calendar attributes such as year, quarter, month, month name, day, and day name.

During the transformation stage, the source data was also checked for matching and uniqueness issues. An important issue was found in the ProductID field because some Product IDs were associated with different Product Names. Therefore, product matching was performed using both Product_ID and Product_Name.

After creating and populating the dimension tables, the transaction data was loaded into the FactSales table.

The fact table stores:

Transaction identifiers.

Foreign keys to the dimension tables.

Ship date.

Sales.

Quantity.

Discount.

Profit.

The dimension keys were retrieved during the loading process by joining the raw data with the corresponding dimension tables.

The final FactSales table contains **2,323 rows**, matching the number of rows in the source data.

Validation

After the loading process, several validation checks were performed.

The validation confirmed that:

RawSuperstore contains 2,323 rows.

FactSales contains 2,323 rows.

The source contains 2,323 unique Row IDs.

FactSales contains 2,323 unique Row IDs.

All source transactions were successfully matched to the required dimension records.

Total quantity was preserved between the raw and fact tables.

Sales and profit totals were compared between the raw and fact layers.

This validation process helped ensure that the transformation and loading stages did not result in missing transaction records.

Final ETL Flow

Central_Superstore.xlsx



CSV



RawSuperstore



Transform



Customer Products Locations Dates ShipMode



FactSales



SQL Analytics



Business Insights

This ETL process provides a clear separation between the original raw data, the structured data warehouse, and the final analytical layer.

8. Database Tables and Warehouse Size

The final SQL Server database contains one raw data table, five dimension tables, and one fact table.

The database was designed to separate the original source data from the analytical warehouse structure.

Raw Data Table

RawSuperstore

RawSuperstore is the staging/raw table that contains the imported source data. It preserves the original transaction-level information and is used as a reference for validation.

Rows: 2,323

Columns: 21

Dimension Tables

Customer

Stores customer information and customer segmentation.

Rows: 629

Primary Key: CustomerKey

Products

Stores product information including category and sub-category.

Rows: 1,326

Primary Key: ProductKey

The number of rows is higher than the 1,310 distinct Product IDs in the source data because some Product IDs appeared with different Product Names.

Locations

Stores geographical information such as country, city, state, postal code, and region.

Rows: 195

Primary Key: LocationKey

Dates

Stores calendar information used for time-based analysis.

Rows: 720

Primary Key: DateKey

ShipMode

Stores the available shipping methods.

Rows: 4

Primary Key: ShipModeKey

Fact Table

FactSales

The central fact table stores the transaction-level sales data and connects the transactions to the dimension tables using foreign keys.

Rows: 2,323

Primary Key: SalesKey

Final Database Summary

Table	Type	Rows
RawSuperstore	Raw / Staging	2,323
Customer	Dimension	629
Products	Dimension	1,326
Locations	Dimension	195
Dates	Dimension	720
ShipMode	Dimension	4
FactSales	Fact	2,323

The final database structure provides a clear separation between raw data, descriptive dimensions, and transactional facts. This makes the database suitable for analytical queries and business reporting.

9. SQL Business Analytics

After building the data warehouse, SQL queries were used to analyze the data and extract meaningful business insights.

The analytical queries were designed to answer different business questions related to sales, profitability, customers, products, time, geography, discounts, and shipping performance.

The analysis also demonstrates several SQL techniques, including aggregations, joins, subqueries, CTEs, CASE expressions, window functions, ranking, and self joins.

9.1 Sales Performance Overview

The first analysis calculates the main business KPIs:

Total Sales

Total Profit

Total Orders

Average Order Value

Profit Margin

The results were:

KPI	Result
Total Sales	501,239.88
Total Profit	39,706.45
Total Orders	1,175
Average Order Value	426.59
Profit Margin	7.92%

These KPIs provide an overall view of the financial performance of the business.

9.2 Sales and Profit by Category

The analysis compares the three main product categories.

Category	Sales	Profit
Technology	170,416.29	33,697.60
Office Supplies	167,026.33	8,879.95
Furniture	163,797.26	-2,871.10

Technology generated the highest sales and the highest profit.

Furniture generated significant sales but resulted in an overall loss. This demonstrates that high sales volume does not necessarily mean high profitability.

9.3 Sales and Profit by Customer Segment

The customer segments were analyzed based on total sales and profit.

Segment	Sales	Profit
Consumer	252,031.43	8,564.05
Corporate	157,995.79	18,703.90
Home Office	91,212.66	12,438.50

Consumer customers generated the highest sales, while Corporate customers generated substantially higher profit despite having lower sales.

This indicates that customer revenue and customer profitability should be analyzed separately.

9.4 Top Customers

The top customers were identified based on total sales.

The highest-selling customer was **Tamara Chand**, with approximately **18,437.14** in sales and **8,745.06** in profit.

Other high-value customers included Adrian Barton, Becky Martin, Sanjit Chand, Harry Marie, and Helen Wasserman.

The analysis shows that some customers generate significantly more revenue than others, making them important customers for the business.

9.5 Top Products by Sales

The analysis identified the products with the highest total sales.

The top-selling product was:

Canon imageCLASS 2200 Advanced Copier

with approximately **17,499.95** in sales and **8,399.98** in profit.

However, some other high-sales products generated negative profit.

This demonstrates an important business point:

High Sales ≠ High Profit

Therefore, product performance should be evaluated using both revenue and profitability.

9.6 Loss-Making Products

Products with negative total profit were identified using the HAVING clause.

Some of the most significant loss-making products included:

GBC DocuBind P400 Electric Binding System

Fellowes PB500 Electric Punch Plastic Comb Binding Machine

3.6 Cubic Foot Counter Height Office Refrigerator

Lexmark MX611dhe Monochrome Laser Printer

Ibico Hi-Tech Manual Binding System

The analysis highlights products that may require further investigation regarding pricing, discounts, costs, or product strategy.

9.7 Monthly Sales and Profit Trend

Monthly sales and profit were analyzed from 2013 through 2016.

The analysis showed significant differences between monthly sales and profitability.

For example, **September 2013** recorded the highest monthly sales at approximately **34,408.69**, while **October 2015** recorded the highest monthly profit at approximately **10,656.87**.

Some months also generated high sales while producing negative profit.

This demonstrates the importance of analyzing both sales and profit when evaluating business performance over time.

9.8 Yearly Performance

Yearly sales, profit, and order counts were analyzed.

Year	Sales	Profit	Orders
2013	103,838.16	539.57	230
2014	102,874.24	11,716.78	234
2015	147,429.42	19,899.19	305
2016	147,098.06	7,550.91	406

2015 was the strongest year in terms of total profit.

In 2016, sales remained almost identical to 2015, but profit decreased significantly despite having more orders.

This is another example of why sales growth does not necessarily translate into profit growth.

9.9 Customers Above Average Sales

A subquery was used to calculate the average sales per customer and identify customers whose total sales were above that average.

This analysis helps identify high-value customers without limiting the results to a fixed number such as the top 10.

The query demonstrates the use of:

Subqueries

Aggregation

HAVING

Customer-level analysis

9.10 Customer Sales Segmentation

A CASE expression was used to classify customers into three groups based on their total sales:

High Value: Sales $\geq$ 5,000

Medium Value: Sales between 2,000 and 4,999.99

Low Value: Sales below 2,000

These thresholds were created specifically for this analysis and are analytical categories rather than values provided by the source dataset.

This segmentation converts numerical sales data into business-friendly customer groups.

9.11 Sub-Category Analysis

Sales, profit, and profit margin were analyzed across product sub-categories.

Some important results included:

Chairs: highest sales at approximately 85,230.68.

Phones: strong profit margin of approximately 17.02%.

Copiers: approximately 41.89% profit margin.

Paper: approximately 39.86% profit margin.

Furnishings: lowest profit margin at approximately -25.61%.

Appliances: approximately -11.19% profit margin.

This analysis demonstrates that high sales volume and high profit margin are not always found in the same product category.

9.12 Category Performance vs Average

The average sales across the three categories were calculated using a window function.

The average category sales were approximately **167,079.96**.

Technology was the only category above the average, with sales of approximately **170,416.29**.

Office Supplies and Furniture were slightly below the average.

9.13 Discount Impact Analysis

Transactions were classified into four discount levels:

No Discount

Low Discount

Medium Discount

High Discount

The results showed a strong relationship between higher discount levels and lower profitability.

For example:

Discount Level	Sales	Profit	Profit Margin
No Discount	243,150.64	76,125.45	31.31%
Low Discount	128,955.12	15,973.43	12.39%
Medium Discount	98,975.04	-11,598.90	-11.72%
High Discount	30,159.08	-40,793.53	-135.26%

The results show that higher discount levels were associated with significantly lower profitability.

However, this analysis indicates an **association rather than causation**, because additional factors may influence profitability.

9.14 Customer Order Frequency

Customer order frequency was analyzed using the number of distinct orders per customer.

The analysis showed that customers with more orders do not necessarily generate the highest sales.

For example, Steve Chapman had 7 orders but approximately 1,937.60 in sales, while Tamara Chand had only 2 orders but approximately 18,437.14 in sales.

This demonstrates that customer frequency and customer value are two different business metrics.

9.15 Repeat vs One-Time Customers

Customers were classified into two groups based on their number of orders.

Customer Type	Customers	Orders	Sales
Repeat Customer	345	891	364,752.73
One-Time Customer	284	284	136,487.15

Repeat customers generated approximately **73% of total sales**, despite representing only about **55% of the customer base**.

This highlights the importance of customer retention and repeat purchases.

9.16 Shipping Performance

Shipping performance was analyzed at the order level to avoid counting individual transaction lines as separate orders.

Ship Mode	Orders	Average Shipping Days
Same Day	62	0.05
First Class	174	2.23
Second Class	224	3.37
Standard Class	715	4.99

Same Day shipping was the fastest method, while Standard Class had the longest average shipping duration.

The analysis used DATEDIFF() to calculate the number of days between order date and ship date.

9.17 Geographic Analysis

Sales and profitability were analyzed by city and state.

Houston recorded the highest city-level sales at approximately **64,504.71**, but generated a loss of approximately **10,153.48**.

Detroit generated approximately **42,446.94** in sales and **13,181.79** in profit.

At the state level, Texas generated the highest sales at approximately **170,187.98**, but recorded a loss of approximately **25,729.29**.

Michigan generated approximately **76,269.61** in sales and **24,463.15** in profit.

This analysis clearly demonstrates that high sales do not necessarily indicate strong financial performance.

9.18 State Profitability Ranking

The RANK() window function was used to rank states according to total profit.

Michigan ranked **1st** in profitability, while Texas ranked **13th** despite having the highest sales.

This ranking provides a clearer view of geographical profitability than sales ranking alone.

9.19 State Contribution to Total Sales

A window function was used to calculate each state's contribution to overall sales.

Texas contributed approximately **33.95%** of total sales.

The four largest states by sales — Texas, Illinois, Michigan, and Indiana — contributed approximately **75.85%** of total sales.

This indicates that a significant portion of revenue is concentrated in a small number of states.

9.20 High-Sales but Low-Profit Products

Products with sales of at least 3,000 but negative total profit were identified.

Examples include:

Lexmark MX611dhe Monochrome Laser Printer

GBC DocuBind P400 Electric Binding System

Fellowes PB500 Electric Punch Plastic Comb Binding Machine

High Speed Automatic Electric Letter Opener

These products generate meaningful revenue but negatively affect profitability and may require further business investigation.

9.21 Cumulative Monthly Sales

A cumulative sales analysis was performed using the SUM() OVER() window function.

The cumulative calculation starts with the first month in the dataset and continuously adds each month's sales.

The final cumulative sales value for December 2016 reached approximately **501,239.88**, matching the overall sales total from the warehouse.

This provides a way to track how total revenue accumulated over time.

9.22 Year-over-Year Sales Comparison

Yearly sales were compared using a Common Table Expression and a self join.

The results showed:

2014 decreased by approximately **0.93%** compared with 2013.

2015 increased by approximately **43.31%** compared with 2014.

2016 decreased by only approximately **0.22%** compared with 2015.

The analysis shows strong sales growth in 2015 followed by relatively stable sales in 2016.

10. SQL Concepts Demonstrated

This project demonstrates a range of SQL concepts and techniques used to build the data warehouse and perform business analytics.

10.1 Basic SQL Queries

The project uses the fundamental SQL clauses required to retrieve and analyze data, including:

SELECT

FROM

WHERE

GROUP BY

ORDER BY

HAVING

These clauses were used throughout the project to filter, group, sort, and summarize the sales data.

10.2 Aggregate Functions

Aggregate functions were used to calculate business metrics from the transaction data.

The main functions used include:

SUM() — calculating total sales, profit, and quantity.

COUNT() — counting records and orders.

COUNT(DISTINCT) — counting unique customers and orders.

AVG() — calculating average values such as average order value and shipping time.

MIN() — finding minimum values.

MAX() — finding maximum values.

These functions were essential for converting transaction-level data into meaningful business metrics.

10.3 JOIN Operations

JOIN operations were used to combine the FactSales table with the dimension tables.

For example, FactSales contains CustomerKey, but the customer name and segment are stored in the Customer table.

Therefore, the two tables are connected using:

FactSales.CustomerKey = Customer.CustomerKey

Similar joins were used with Products, Locations, Dates, and ShipMode.

This allowed the analysis to combine transaction measures with descriptive business information.

10.4 GROUP BY

GROUP BY was used to create business-level summaries.

Examples include:

Sales by category.

Sales by customer.

Sales by state.

Sales by year.

Sales by month.

Sales by shipping mode.

For example, grouping by Category allows total sales and profit to be calculated separately for Technology, Office Supplies, and Furniture.

10.5 HAVING

The HAVING clause was used to filter aggregated results.

For example, loss-making products were identified using:

HAVING SUM(Profit) < 0

Unlike WHERE, which filters individual rows before aggregation, HAVING filters groups after aggregation.

10.6 CASE Expressions

CASE expressions were used to convert numerical values into meaningful business categories.

For example, customers were classified as:

High Value

Medium Value

Low Value

based on their total sales.

CASE was also used to classify transactions according to discount levels.

This makes analytical results easier to interpret from a business perspective.

10.7 Subqueries

Subqueries were used when the result of one query was required as part of another analysis.

One example was identifying customers whose total sales were above the average customer sales.

The inner query calculated sales for each customer, while another level calculated the average of those customer totals.

This demonstrated how subqueries can be used to solve multi-step analytical problems.

10.8 Common Table Expressions (CTEs)

Common Table Expressions were used to make complex queries easier to organize and understand.

CTEs were used in analyses such as:

Sub-category performance.

Repeat versus one-time customers.

Shipping performance.

Cumulative sales.

Year-over-year sales comparison.

A CTE allows an intermediate result to be defined first and then used by the main query.

10.9 Window Functions

Window functions were used to perform calculations across groups of rows without collapsing the result into a single row.

Examples include:

SUM() OVER()

and

RANK() OVER()

They were used for:

Calculating average category sales.

Calculating contribution to total sales.

Calculating cumulative sales.

Ranking states by profitability.

10.10 Ranking

The RANK() window function was used to rank states based on total profit.

This allowed every state to receive a profitability position instead of only displaying a limited number of states.

For example, Michigan ranked first in total profit while Texas ranked thirteenth.

10.11 Self Join

A self join was used for the yearly sales comparison.

The yearly sales result was treated as two logical copies:

Current Year

Previous Year

The two results were then joined based on consecutive years to calculate sales growth and growth percentage.

This approach was used instead of LAG() because the project focused on SQL techniques that had already been studied.

10.12 NULLIF

NULLIF() was used when calculating percentages and profit margins.

For example:

Profit / NULLIF(Sales, 0)

This prevents division-by-zero errors when the denominator is zero.

10.13 Views

A reusable SQL View named:

vw_SalesKPIs

was created to provide the main business KPIs, including:

Total Sales

Total Profit

Total Orders

Average Order Value

Profit Margin

The View allows the KPI calculation to be reused without rewriting the complete query.

10.14 Stored Procedures

A Stored Procedure named:

sp_SalesAnalysis

was created to perform sales analysis based on two parameters:

Year

Category

For example, the procedure can be executed for:

2015 + Technology

and returns the corresponding sales, profit, quantity, orders, and profit margin.

This demonstrates how SQL Server can be used to create reusable and parameterized analytical operations.

10.15 Primary Keys and Foreign Keys

Primary keys were used to uniquely identify records in the dimension and fact tables.

Foreign keys were used in FactSales to establish relationships with the dimension tables.

This ensures that the data warehouse follows a structured relational design and maintains referential integrity.

Summary

The project demonstrates how SQL can be used beyond basic data retrieval. The combination of relational modeling, aggregation, joins, subqueries, CTEs, CASE, window functions, ranking, views, stored procedures, and validation creates a complete analytical workflow from raw data to business insights.

11. Key Business Insights

The analysis of the Central Superstore dataset revealed several important business insights related to sales, profitability, customers, products, discounts, geography, and shipping performance.

11.1 Sales and Profitability

The business generated approximately **501,239.88** in total sales and **39,706.45** in total profit, resulting in an overall profit margin of approximately **7.92%**.

The results show that sales volume alone is not enough to evaluate business performance. Several products, cities, and states generated high sales while producing low or negative profit.

11.2 Category Performance

Technology was the strongest overall category, generating approximately **170,416.29** in sales and **33,697.60** in profit.

Office Supplies generated similar sales at approximately **167,026.33**, but its profit was significantly lower at **8,879.95**.

Furniture generated approximately **163,797.26** in sales but recorded a loss of **2,871.10**.

This indicates that Furniture requires further investigation into pricing, discounts, costs, and product mix.

11.3 Customer Profitability

Consumer customers generated the highest sales at approximately **252,031.43**.

However, Corporate customers generated approximately **18,703.90** in profit compared with only **8,564.05** from Consumer customers.

This shows that the segment with the highest sales is not necessarily the most profitable segment.

11.4 Repeat Customers

Repeat customers represented **345 customers**, compared with **284 one-time customers**.

Although repeat customers represented approximately **55%** of the customer base, they generated approximately **364,752.73** in sales, which represents about **73% of total sales**.

This highlights the importance of customer retention and repeat purchases.

11.5 Product Profitability

Several high-selling products were found to have negative profit.

For example, the **Lexmark MX611dhe Monochrome Laser Printer** generated approximately **14,279.91** in sales but produced a loss of approximately **1,189.99**.

The **GBC DocuBind P400 Electric Binding System** generated approximately **8,710.33** in sales but produced a loss of approximately **3,048.62**.

These products should be reviewed because they generate revenue but negatively affect profitability.

11.6 Discount and Profitability

The discount analysis showed a strong association between higher discount levels and lower profitability.

Transactions without discounts generated a profit margin of approximately **31.31%**, while high-discount transactions produced a negative margin of approximately **-135.26%**.

This suggests that aggressive discounting can be associated with significant profitability problems.

However, the analysis shows an association rather than proving that discounts alone caused the losses.

11.7 Geographic Performance

Texas generated the highest sales among the states, with approximately **170,187.98** in sales.

However, Texas also generated a loss of approximately **25,729.29**.

In contrast, Michigan generated approximately **76,269.61** in sales and **24,463.15** in profit, ranking first in total state profit.

This demonstrates why geographic performance should be evaluated using both sales and profitability.

11.8 Sales Concentration

Texas contributed approximately **33.95%** of total sales.

The four largest states by sales — Texas, Illinois, Michigan, and Indiana — contributed approximately **75.85%** of total sales.

This indicates that a significant portion of the business revenue is concentrated in a relatively small number of states.

11.9 City Performance

Houston recorded the highest city-level sales at approximately **64,504.71**, but generated a loss of approximately **10,153.48**.

Detroit generated lower sales at approximately **42,446.94**, but produced approximately **13,181.79** in profit.

This provides another example of the difference between revenue performance and profitability.

11.10 Yearly Performance

2015 was the strongest year in terms of total profit, generating approximately **19,899.19**.

In 2016, sales remained almost unchanged compared with 2015:

2015 Sales: **147,429.42**

2016 Sales: **147,098.06**

However, profit decreased from **19,899.19** to **7,550.91**.

This indicates that maintaining sales levels does not necessarily mean maintaining profitability.

11.11 Shipping Performance

Shipping performance followed a clear pattern.

Same Day shipping had the shortest average shipping time at approximately **0.05 days**, while Standard Class had the longest average at approximately **4.99 days**.

This analysis provides a basic operational view of shipping performance across the different shipping methods.

11.12 Overall Business Conclusion

The overall analysis shows that the most important business lesson from this dataset is:

Revenue alone is not enough to evaluate business performance.

Sales should be analyzed together with:

Profit

Profit Margin

Customer Value

Product Performance

Discounts

Geographic Performance

Order Frequency

Shipping Performance

Combining these metrics provides a more complete view of business performance and helps identify areas that require further investigation.

12. Views and Stored Procedures

To make the analytical work reusable, two database objects were created in SQL Server: a **View** for the main business KPIs and a **Stored Procedure** for parameterized sales analysis.

12.1 Sales KPI View

A SQL View named vw_SalesKPIs was created to provide a reusable summary of the main business performance indicators.

The View calculates:

Total Sales

Total Profit

Total Orders

Average Order Value

Profit Margin

The View was tested successfully and returned:

KPI	Result
Total Sales	501,239.88
Total Profit	39,706.45
Total Orders	1,175
Average Order Value	426.59
Profit Margin	7.92%

The main advantage of the View is that the KPI calculations can be reused without rewriting the complete aggregation query.

It can be accessed using:

```
SELECT *  
  
FROM vw_SalesKPIs;
```

12.2 Sales Analysis Stored Procedure

A Stored Procedure named sp_SalesAnalysis was created to perform a reusable sales analysis based on two parameters:

@Year

@Category

The procedure returns:

Year

Category

Total Sales

Total Profit

Total Quantity

Total Orders

Profit Margin

For example, the procedure was tested using:

EXEC sp_SalesAnalysis

@Year = 2015,

@Category = 'Technology';

The result was:

Year	Category	Sales	Profit	Quantity	Orders	Profit Margin
2015	Technology	50,863.43	13,760.55	465	102	27.05%

The Stored Procedure demonstrates how SQL Server can be used to create reusable and parameterized analytical operations.

Instead of writing a new query for every year and category combination, the user can simply provide different parameter values.

For example, the same procedure can be reused for:

2014 + Furniture

2015 + Technology

2016 + Office Supplies

This makes the analytical layer more flexible and easier to reuse.

12.3 Why These Objects Were Created

The View and Stored Procedure demonstrate two different approaches to reusable SQL logic:

View: provides a reusable result set for the main business KPIs.

Stored Procedure: provides parameterized analysis based on user-selected inputs.

Together, they make the SQL project more practical and demonstrate how analytical logic can be transformed into reusable database objects.

13. Data Validation

Data validation was performed after loading the data into the SQL Server data warehouse to make sure that the transformation and loading process did not cause data loss or duplicate transaction records.

13.1 Row Count Validation

The number of rows in the raw data was compared with the number of rows in the final fact table.

The validation returned:

Table	Rows
RawSuperstore	2,323
FactSales	2,323

This confirms that all source transaction rows were successfully loaded into the FactSales table.

13.2 Row ID Validation

The number of unique Row IDs was also compared between the raw and fact tables.

Validation	Result
Raw Unique Row IDs	2,323
Fact Unique Row IDs	2,323

The matching results confirm that there were no duplicated or missing transaction Row IDs during the ETL process.

13.3 Order Validation

The final FactSales table contains:

2,323 transaction rows

1,175 unique orders

8,780 total quantity

The difference between transaction rows and orders is expected because one order can contain multiple products and therefore multiple transaction lines.

13.4 Financial Validation

The total Sales, Profit, and Quantity values from the raw data were compared with the final FactSales table.

Measure	RawSuperstore	FactSales
Sales	501,239.89	501,239.88
Profit	39,706.36	39,706.45
Quantity	8,780	8,780

The results are almost identical, with only very small differences in Sales and Profit caused by numeric precision during the data import and transformation process.

Quantity matched exactly between the two tables.

13.5 Dimension Matching

All 2,323 source transactions were successfully matched with the required dimension records:

Customer

Products

Locations

Dates

ShipMode

A special validation was required for the Products dimension because some source Product IDs appeared with different Product Names.

Therefore, product matching was performed using both:

Product_ID + Product_Name

instead of using Product ID alone.

This ensured that all 2,323 transaction records were correctly connected to the appropriate product records.

13.6 Validation Conclusion

The validation process confirmed that:

All source transaction rows were loaded.

All Row IDs remained unique.

All 2,323 transactions were successfully matched with the required dimensions.

The total quantity was preserved exactly.

Sales and Profit totals remained effectively consistent.

The final Star Schema maintained the required relationships between the fact and dimension tables.

These checks provide confidence that the final data warehouse is suitable for the analytical queries and business insights presented in this project.

14. Tools and Technologies

The project was developed using a combination of data preparation, database management, and analytical technologies.

14.1 Microsoft Excel

Microsoft Excel was used as the original source of the Superstore dataset.

The Excel workbook contained the transaction-level sales data used throughout the project.

14.2 CSV

The source Excel sheet was converted into a UTF-8 CSV file before importing it into SQL Server.

The CSV format was used because the required Microsoft Excel database provider was not available in the SQL Server environment.

14.3 Microsoft SQL Server

SQL Server was used as the main database management system for the project.

It was used to:

Store the raw source data.

Create the data warehouse tables.

Define primary and foreign keys.

Load and transform the data.

Perform analytical queries.

Create Views.

Create Stored Procedures.

Validate the final data.

14.4 SQL Server Management Studio (SSMS)

SQL Server Management Studio was used as the main development environment for writing and executing SQL queries.

SSMS was used for:

Database creation.

Table creation.

Data loading.

Data validation.

Business analysis.

View creation.

Stored Procedure creation.

Testing SQL results.

14.5 SQL

SQL was the main analytical language used in the project.

The project demonstrated several SQL concepts, including:

SELECT statements

Filtering with WHERE

GROUP BY

ORDER BY

HAVING

Aggregate functions

INNER JOIN

LEFT JOIN

Subqueries

Common Table Expressions (CTEs)

CASE statements

Window Functions

RANK()

SUM() OVER()

Self Joins

NULLIF()

Views

Stored Procedures

Primary Keys

Foreign Keys

14.6 Project Architecture

The overall technology workflow can be summarized as:

Microsoft Excel



CSV



SQL Server / SSMS



Raw Data Layer



Star Schema Data Warehouse



SQL Analytics



Business Insights

14.7 Main Technologies Summary

Technology	Purpose
Microsoft Excel	Original data source
CSV	Data transfer and import format
Microsoft SQL Server	Database and data warehouse
SQL Server Management Studio	SQL development and database management
SQL	Data transformation and business analytics

The combination of these technologies allowed the project to move from raw transaction data to a structured analytical data warehouse and finally to business-oriented insights.

15. Conclusion

This project demonstrated the complete process of transforming raw sales data into a structured SQL Server data warehouse and using SQL to generate meaningful business insights.

The project started with a transaction-level Superstore dataset containing 2,323 records from the Central region. The raw data was imported into SQL Server and organized into a Star Schema consisting of dimension tables and a central FactSales table.

The data warehouse design improved the structure of the original dataset by separating customers, products, locations, dates, and shipping methods from the sales transactions.

SQL was then used to perform different types of business analysis, including:

Overall sales and profitability analysis.

Category and sub-category performance.

Customer value and order frequency analysis.

Product profitability analysis.

Discount impact analysis.

Monthly and yearly sales trends.

Geographic performance by city and state.

Shipping performance.

Customer retention analysis.

Ranking and contribution analysis.

The project also demonstrated advanced SQL techniques such as CTEs, subqueries, CASE statements, window functions, self joins, Views, and Stored Procedures.

The analysis showed that sales revenue alone is not enough to evaluate business performance. Several areas with high sales also generated low or negative profits. For example, Texas generated the highest sales among the states but also recorded the largest overall loss. Similarly, some high-selling products were found to be loss-making.

The project also showed the importance of customer behavior. Repeat customers represented more than half of the customer base and generated approximately 73% of total sales, highlighting their importance to overall business performance.

Overall, this project demonstrates how SQL can be used not only to retrieve data, but also to build a structured analytical environment and support business decision-making.

The final result is a complete SQL-based analytics project that combines **data warehousing, ETL, database design, SQL analysis, validation, and business insights** in one practical workflow.

16. Project Structure and File Organization

The project files are organized into separate folders based on their purpose. This structure makes the project easier to understand, maintain, and reuse.

The recommended project structure is:

```
Central-Superstore-SQL-Project/  
|  
├── README.md  
|  
├── SQL/  
|   ├── 01_Database_Setup.sql  
|   ├── 02_Create_Tables.sql  
|   ├── 03_Load_Data.sql  
|   ├── 04_Analytics.sql  
|   ├── 05_Views.sql  
|   └── 06_Stored_Procedures.sql  
|  
├── Data/  
|   └── Central_Superstore.csv  
|  
├── Documentation/  
|   └── Central_Superstore_SQL_Project_Documentation.pdf  
|  
└── Screenshots/
```

16.1 README.md

The README file provides a short overview of the project and explains:

Project objective.

Dataset information.

Data warehouse architecture.

Main SQL analysis.

Key business insights.

Technologies used.

Project structure.

It serves as the first introduction to the project when viewed on GitHub.

16.2 SQL Folder

The SQL folder contains the SQL scripts used to build and analyze the project.

The scripts are separated according to their purpose:

01_Database_Setup.sql — Creates the database.

02_Create_Tables.sql — Creates the raw, dimension, and fact tables.

03_Load_Data.sql — Loads and transforms the data.

04_Analytics.sql — Contains the business analysis queries.

05_Views.sql — Contains reusable SQL Views.

06_Stored_Procedures.sql — Contains Stored Procedures.

16.3 Data Folder

The Data folder contains the source dataset used for the project.

The dataset is stored as:

Central_Superstore.csv

16.4 Documentation Folder

The Documentation folder contains the complete project documentation.

The documentation describes the project from the initial dataset and data preparation stages through the data warehouse design, SQL analysis, validation, and final business insights.

16.5 Screenshots Folder

The Screenshots folder can contain selected screenshots demonstrating:

Database structure.

Star Schema.

SQL queries and results.

KPI results.

View execution.

Stored Procedure execution.

Important analytical findings.

Screenshots are included to provide visual evidence of the completed project.

16.6 Final Project Organization

The final structure separates the project into four main areas:

SQL Code



Data



Documentation



Screenshots

This organization makes the project easier for reviewers, recruiters, and other users to navigate and understand.